Preinforme – Desafío II

Diseño e Implementación de la Red Social Ude@Book en C++

Sofía Arboleda Zapata
Jose Manuel Arias Toro
UdeA

09 de octubre de 2026

1. Contextualización y Análisis del Problema

El presente proyecto abarca el diseño conceptual y la arquitectura orientada a objetos para la red social Ude@Book. El sistema gestionará eficientemente la información de usuarios, amistades, álbumes, fotos y mensajes, operando estrictamente bajo el paradigma de Programación Orientada a Objetos (POO) en C++.

El desafío central radica en establecer relaciones entre objetos (como amistades y mensajes) minimizando la duplicación de datos en el *Heap* y garantizando una carga coherente desde el almacenamiento permanente.

2. Consideraciones de Implementación y Eficiencia

Para asegurar un diseño robusto y de bajo consumo de recursos, establecimos la siguiente idea para el diseño:

- **Mensajes centralizados (Un solo dueño por mensaje):** Con la idea de no desperdiciar memoria guardando el mismo texto dos veces (una copia para el que envía y otra para el que recibe), cada mensaje se creará una sola vez en la memoria dinámica. El truco está en definir un "dueño": el usuario que envía el mensaje será el único responsable de crearlo (`new`) y de destruirlo (`delete`). El que lo recibe simplemente guardará una referencia (un puntero) para poder leerlo. Así, nos ahorramos la mitad de la memoria en los chats y evitamos que el programa colapse por intentar borrar el mismo mensaje dos veces.
- **Amistades por índices y Matriz 1D:** En lugar de guardar las direcciones de memoria (`punteros`) de los amigos (las cuales cambian cada vez que se cierra y se abre el programa), vamos a identificarlos simplemente por su número de posición (índice) dentro del arreglo general de usuarios. Esto nos facilita muchísimo la vida a la hora de guardar y cargar los datos desde los archivos de texto. Además, esta lógica encaja perfecto con la idea de usar una matriz de adyacencia aplanada (de una sola dimensión) llena de unos y ceros, haciendo que calcular los amigos en común para las sugerencias sea un proceso súper rápido y directo.

3. Diseño y Arquitectura de la Solución

El proyecto está dividido de forma muy organizada (modular) para que el código no sea más legible y cada clase tenga una tarea clara y específica:

- `Fecha` y `Foto`: Son las clases más básicas. `Fecha` se encarga de manejar el tiempo y nos permite comparar fácilmente qué evento pasó antes o después (algo clave para ordenar los mensajes). Por su lado, `Foto` simplemente guarda la información de cada imagen, como cuántos likes tiene y su ruta en el disco.

- `Album` y `Mensaje`: `Album` funciona como una carpeta que guarda un arreglo de fotos y nos permite navegar por ellas en forma de carrusel (pasando a la siguiente o a la anterior, y al llegar al final vuleve a la primera). `Mensaje` es la clase que guarda el texto, sabe quién lo envió, a quién va dirigido y nos dice si ya fue leído o no.
- `Usuario`: Es el corazón de los datos. Esta clase agrupa toda la información personal de alguien, guarda sus álbumes, mantiene la lista de sus amigos (usando los índices que mencionamos arriba) y tiene a la mano los mensajes que ha enviado y recibido.
- `Amigos` y `Metricas`: Son nuestras herramientas de apoyo. `Amigos` se dedica exclusivamente a manejar la matriz de adyacencia (haciendo toda la matemática de cruzar índices y buscar amigos en común). `Metricas`, por su parte, es la que está pendiente de ir contando la memoria que gastamos y las iteraciones del programa, para mostrar el reporte al final.
- `UdeBook`: Es como el "jefe"del programa. Es la clase principal que guarda la lista grande con todos los usuarios registrados. Se encarga del trabajo pesado: lee y guarda los datos en los archivos, controla cuando alguien inicia sesión y reparte las tareas a las demás clases cuando el usuario elige una opción en el menú.

3.1. Diagrama de Clases (Resumen)

La Figura 4 ilustra las relaciones y composición principales del sistema.

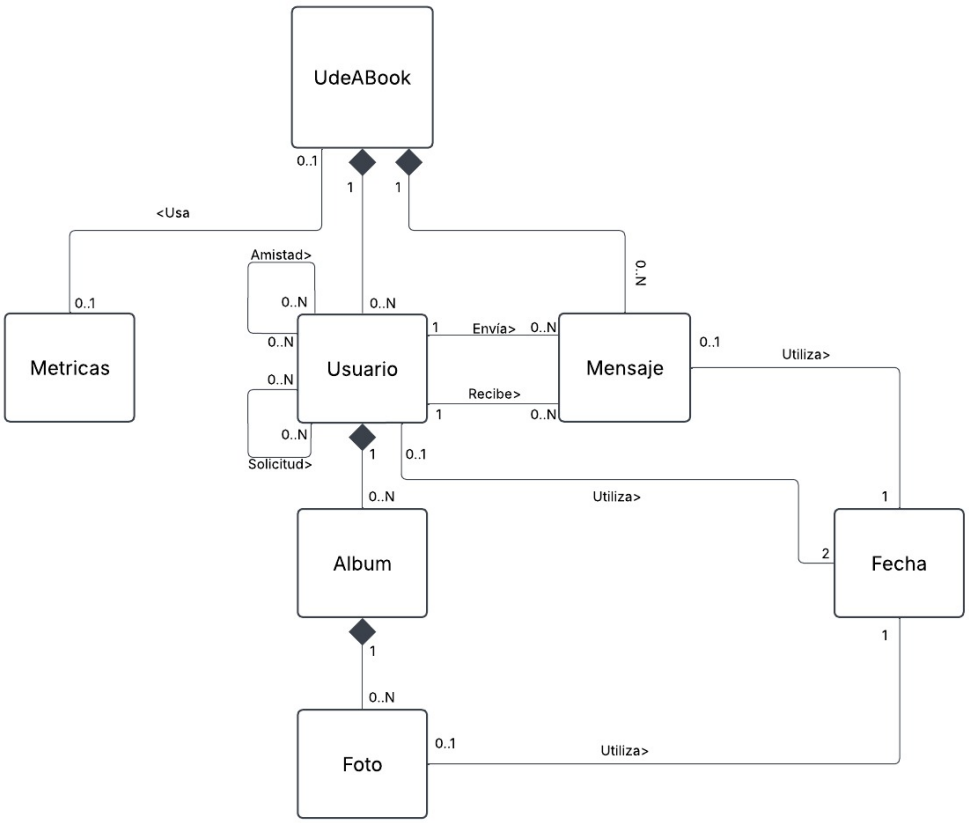


Figura 1: Diagrama de clases resumen

Nota: El detalle de todas las clases, se encuentra acontinuación en el anexo de clases expandidas.

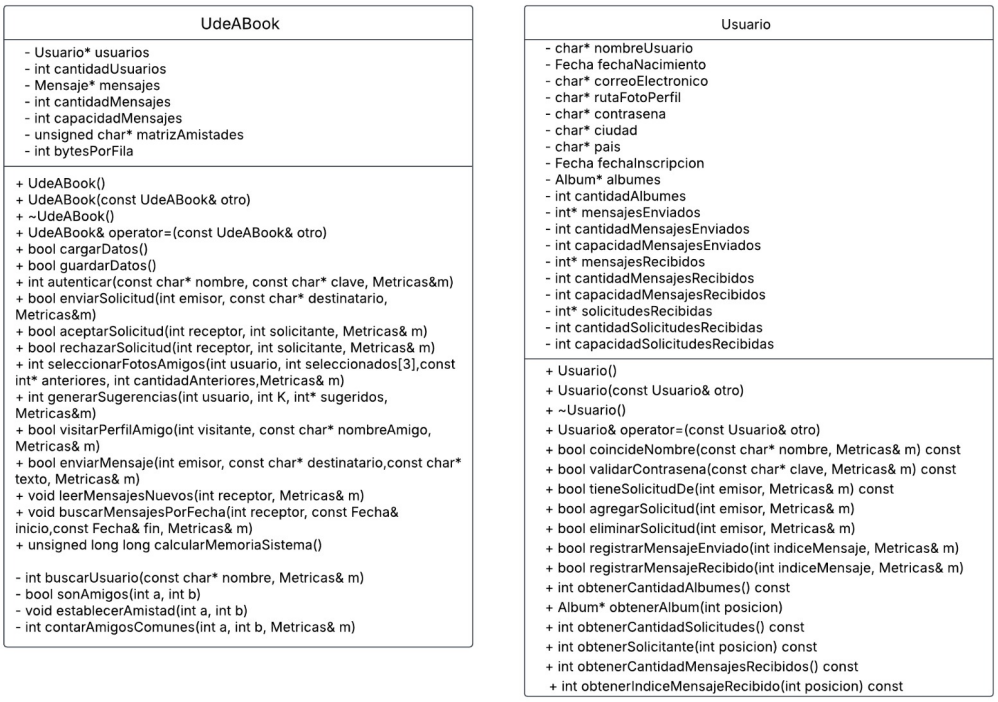


Figura 2: Clases expandidas

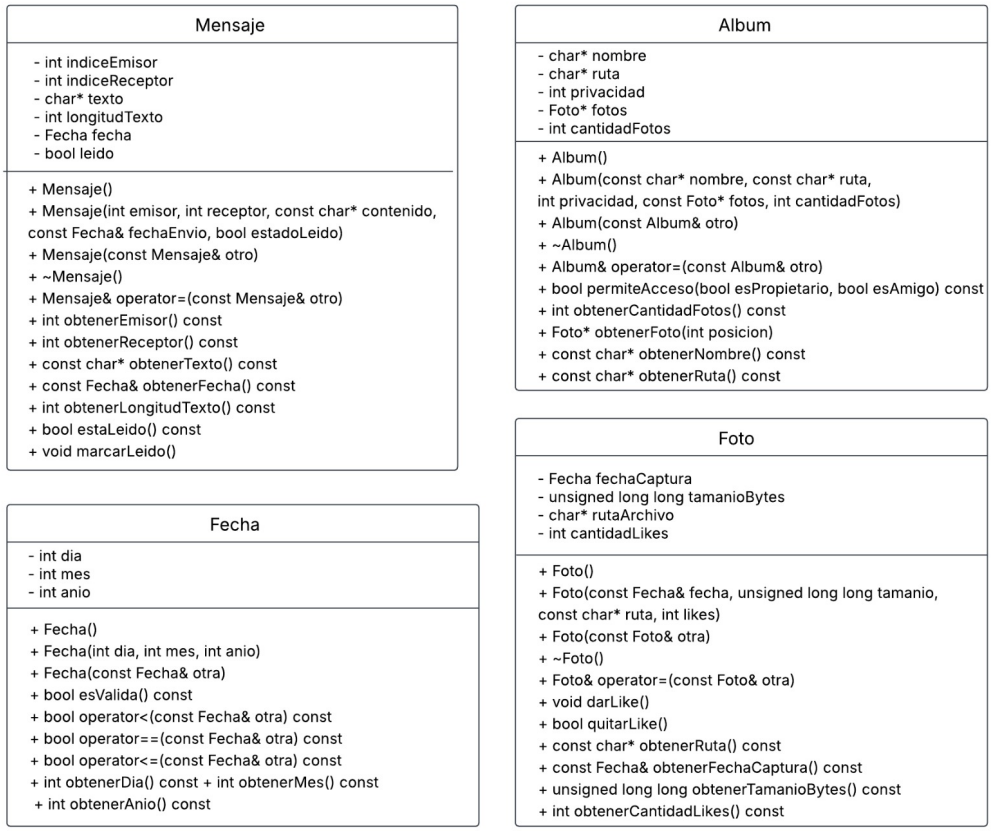


Figura 3: Clases expandidas

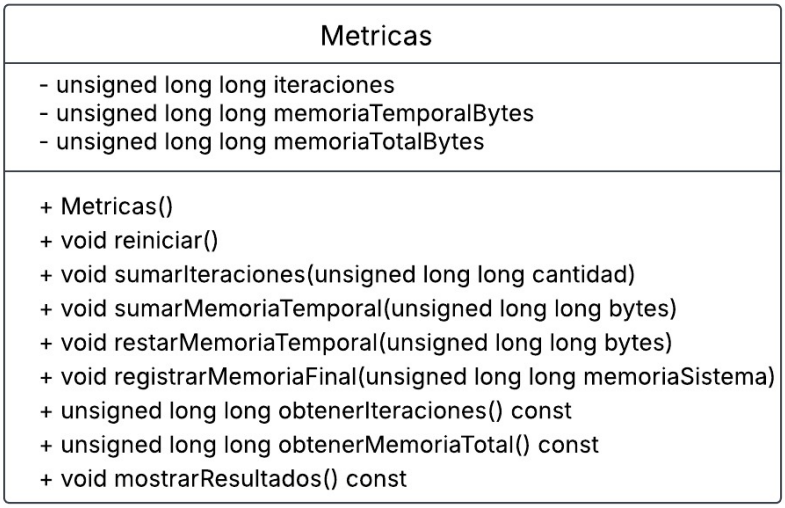


Figura 4: Clases expandidas

Nota: El detalle de todas las clases, se encuentra acontinuación en el anexo de clases expandidas.

4. Algoritmos Esenciales (Lógica de Alto Nivel)

1. **Cargar y guardar los datos (Persistencia):** Cuando el programa arranca, la clase principal (`UdeBook`) lee los archivos de texto para armar toda la red social. Si el archivo dice que el usuario A y el usuario B son amigos, el programa hace dos cosas al mismo tiempo para que todo cuadre: primero, anota un 1 en nuestra matriz de amistades; y segundo, guarda el índice de B en la lista de amigos de A, y el índice de A en la lista de B. Así nos aseguramos de que la amistad funcione en ambos sentidos y no queden datos sueltos.

2. **El algoritmo para sugerir amigos:** Para recomendarle personas a un usuario, le sacamos todo el provecho a nuestra matriz de amistades. Lo que hacemos es tomar la lista de amigos del usuario que inició sesión y compararla con los amigos del resto de personas en la plataforma. Por cada persona nueva que evaluamos, contamos cuántos amigos en común tienen. Al final, organizamos esa lista de mayor a menor coincidencia y le mostramos en pantalla los K perfiles con los que tiene más gente conocida en común.